

Unipile Authentication & Subscription Management — Implementation Plan

Project Beacon · G3 Recruitment Automation Platform

Date: July 31, 2026

Target Architecture: TanStack Start (React 19) · Prisma · PostgreSQL · Cloudflare Nitro

1. Overview: Transitioning from Manual POC to Automated Backend Pipeline

In our initial Proof of Concept (POC), account connections were tested manually by triggering `POST` API calls to Unipile and walking through the auth wizard.

This implementation plan moves that entire workflow into our application's backend. Recruiters can connect their accounts seamlessly with a single click inside the G3 dashboard, while Sundar (Owner) gets complete control over active recruiter seats and subscription costs.

Key Benefits:

- **Zero Credential Custody:** Recruiters authenticate directly via Unipile's Hosted Wizard. G3 never sees, handles, or stores passwords, 2FA codes, or session cookies (`li_at`).
 - **Automated 2-Step Handshake:** Our backend listens to Unipile webhooks to track initial connection sync (`CONNECTING` ) and full sync completion (`OK` ).
 - **Self-Healing Accounts:** If a password changes or 2FA expires, the platform alerts the recruiter with a 1-click **"Reconnect Account"** button (`RECONNECTION_NEEDED` ).
 - **Owner Subscription Control:** Sundar manages total active seats, capping peak billing before link generation ever occurs.
-

2. End-to-End User Workflows

2.1 Recruiter Flow (Connecting & Managing Accounts)

1. **Recruiter Clicks "Connect LinkedIn":** Inside G3 Recruiter Settings, the user selects a channel (LinkedIn, Gmail, Outlook, or WhatsApp) and clicks **Connect**.
 2. **Server Generates Auth Link:** The G3 backend calls Unipile's `POST /api/v1/hosted/accounts/link` with an opaque, single-use security nonce.
 3. **Full-Page Redirect:** G3 redirects the browser to Unipile's hosted wizard. (Note: Never embedded in an iframe, as 2FA and captchas fail in frames).
 4. **Step 1 — Connection Established (`CONNECTING`):** When authentication succeeds, Unipile fires the `notify_url` callback (`CREATION_SUCCESS`). G3 records the mapping and shows a **Blue Badge ("Connecting...")** while background history sync begins.
 5. **Step 2 — Sync Completed (`OK`):** Once background contact/chat history finishes syncing, Unipile's status webhook fires (`SYNC_SUCCESS`). G3 updates the badge to **Green ("Connected")**.
 6. **Reconnection Handling:** If credentials break, Unipile fires `CREDENTIALS` . G3 turns the badge **Red ("Reconnection Needed")** and provides a 1-click reconnect button (`type: reconnect`).
-

2.2 Owner Flow for Sundar / Ethan (Seat & Subscription Management)

1. **Seat Allocation:** Sundar views and updates total active recruiter slots (`unipile.max_slots` , e.g., 10 slots) directly from the Owner Dashboard.
2. **Peak-Based Billing Protection:** Unipile bills based on the **peak number of simultaneously active linked accounts** in a rolling 30-day billing period (€49/month base for 10 accounts, ~€5/account beyond).
3. **Server-Side Enforcement:** Before any recruiter can generate a connection link, G3 checks available slots against `unipile.max_slots` . If the seat limit is reached, link generation is blocked before any Unipile API call or billing event occurs.
4. **Account Removal:** Sundar can disconnect inactive recruiter accounts via `DELETE /api/v1/accounts/{id}` . The dashboard explicitly notes: *"Disconnecting an account frees capacity for future billing cycles."*

3. Simplified Diagrams & Badge Matrix

3.1 Authentication Sequence Diagram

```
sequenceDiagram
    autonumber
    actor Recruiter as Recruiter (Browser)
    participant UI as G3 Dashboard UI
    participant API as G3 Backend Server
    participant DB as PostgreSQL Database
    participant UP as Unipile API

    Recruiter->>UI: Clicks "Connect LinkedIn"
    UI->>API: POST /api/unipile/connect { provider: "LINKEDIN" }
    Note over API: Checks Seat Limit (SystemConfig) & creates 15-min TTL nonce
    API->>DB: Stores UnipileAuthAttempt (nonce)
    API->>UP: POST /api/v1/hosted/accounts/link (name: nonce, single_use: true)
    UP-->>API: Returns Hosted Auth URL { url: "https://account.unipile.com/..." }
    API-->>UI: Returns Auth URL
    UI->>Recruiter: Full-Page Redirect to Unipile Wizard

    Note over Recruiter,UP: Recruiter completes login & 2FA on Unipile

    UP->>API: POST /api/unipile/webhook/$token (notify_url)
    Note over API: Stage 1: CREATION_SUCCESS
    API->>DB: Resolves nonce -> userId; Upserts ConnectedAccount (status: CONNECTING)
    API-->>UP: HTTP 200 OK

    UP->>Recruiter: Redirects back to G3 Settings (?connect=ok)
    UI->>API: GET /api/unipile/accounts
    API-->>UI: Returns ConnectedAccount status ('CONNECTING')
    UI-->>Recruiter: Displays Blue Badge ("Connecting...")

    Note over UP: Unipile syncs message history in background

    UP->>API: POST /api/unipile/webhook/$token (account_status webhook)
    Note over API: Stage 2: SYNC_SUCCESS / OK
    API->>DB: Updates ConnectedAccount (status: OK)
    API-->>UP: HTTP 200 OK

    UI->>API: Refetches account status
    API-->>UI: Returns status ('OK')
    UI-->>Recruiter: Flips badge to Green ("Connected")
```

3.2 Status Normalization & UI Badge Matrix

```
flowchart TD
    A[Unipile Webhook Received] --> B[extractUnipileStatus: Read AccountStatus.message OR status]
    B --> C[normalizeUnipileStatus Helper]

    C -->|CREATION_SUCCESS / CONNECTING| D[CONNECTING: Blue Spinner Badge]
    C -->|OK / SYNC_SUCCESS / RECONNECTED| E[OK: Green Badge 'Connected']
    C -->|CREDENTIALS| F[RECONNECTION_NEEDED: Red Badge 'Reconnection Needed']
    C -->|PERMISSIONS| G[PERMISSION_REVOKED: Red Badge 'Re-grant Access']
    C -->|CREATION_FAIL| H[CONNECT_FAILED: Amber Badge 'Could Not Connect']
    C -->|ERROR / STOPPED / Unmapped| I[SYNC_STOPPED: Amber Badge 'Sync Stopped']
    C -->|DELETED / DISCONNECTED| J[DISCONNECTED: Gray Badge 'Not Connected']

    D --> K[Persist in ConnectedAccount Table]
    E --> K
    F --> K
    G --> K
    H --> K
    I --> K
    J --> K

    K --> L[Render UI Badge on Recruiter & Owner Dashboards]
```

Detailed Badge Matrix

Status Enum	UI Badge Style	Display Copy	Triggering Vendor Event	Recruiter Action
CONNECTING	 Blue (Spinner)	"Connecting... (Syncing history)"	CREATION_SUCCESS , CONNECTING	Wait for background sync
OK	 Green (Solid)	"Connected"	OK , SYNC_SUCCESS , RECONNECTED	Disconnect
RECONNECTION_NEEDED	 Red (Solid)	"Reconnection Needed"	CREDENTIALS	1-Click Reconnect
PERMISSION_REVOKED	 Red (Solid)	"Re-grant Access"	PERMISSIONS	Re-grant OAuth Access
CONNECT_FAILED	 Amber (Solid)	"Could Not Connect"	CREATION_FAIL	Try Again
SYNC_STOPPED	 Amber (Solid)	"Sync Stopped"	ERROR , STOPPED , unmapped strings	Reconnect / Contact Support
DISCONNECTED	 Gray (Outline)	"Not Connected"	DELETED or no row	Connect Account

4. Dashboard Mockups & Component Locations

4.1 Recruiter Dashboard — Integrations Component

Location in Codebase: client/src/routes/recruiter.settings.tsx (or client/src/routes/settings.tsx)

```
+-----+
| ACCOUNT INTEGRATIONS                                     |
| Connect your personal LinkedIn and messaging channels for automated outreach. |
+-----+
| [LinkedIn Icon]  LinkedIn Account                       |
|                  john.doe@example.com (John Doe)       |
|                  Badge: [ ● Connected ]                |
|                  Last Synced: 2 mins ago                |
|                  [ Disconnect ]                         |
+-----+
| [Gmail Icon]     Google Workspace                       |
|                  john.recruiter@gmail.com              |
|                  Badge: [ ● Reconnection Needed ]      |
|                  Reason: Token expired or password changed |
|                  [ ✕ Reconnect Account ]               |
+-----+
| [WhatsApp Icon]  WhatsApp Business                     |
|                  Not Connected                          |
|                  Badge: [ ● Not Connected ]             |
|                  [ Connect WhatsApp ]                  |
+-----+
```

4.2 Owner Dashboard — Sundar's Seat & Subscription Management Panel

Location in Codebase: client/src/routes/owner.settings.tsx (or client/src/routes/admin/seats.tsx)

```
+-----+
| OWNER DASHBOARD – UNIPILE SEAT & SUBSCRIPTION MANAGEMENT |
+-----+
| Active Seats: [ 8 / 10 Used ] =====> (80% Capacity)  [ Edit Seat Cap ] |
| Rolling 30-Day Peak Billing: €49.00 / month (Includes 10 Base Accounts)    |
| Est. Next Billing Cycle: €49.00                                           |
| ⓘ Note: Billing is based on peak simultaneous accounts during the 30-day period. |
|   Deleting an account frees capacity for future billing cycles.           |
+-----+
| CONNECTED RECRUITER ACCOUNTS                                             |
+-----+
| Recruiter Name | Provider | Account | Status | Actions |
+-----+
| Sarah Connor  | LINKEDIN | s.connor | ● Connected | [ Revoke ] |
| Alex Mercer    | LINKEDIN | a.mercer | ● Connecting | [ Revoke ] |
| David Miller   | GOOGLE   | d.miller | ● Reconnect | [ Remind ] |
+-----+
```

5. What To Do Where — Code Base Map

```

server/
├── lib/unipile/
│   ├── client.ts      <-- Typed fetch wrapper (Handles DSN, X-API-KEY, ?port= workaround)
│   ├── status.ts      <-- extractUnipileStatus() & normalizeUnipileStatus()
│   ├── hosted-auth.ts  <-- mintConnectLink() & mintReconnectLink()
│   ├── webhooks.ts     <-- provisionWebhooks() (Registers account_status webhook at deploy)
│   └── handlers.ts     <-- applyAccountStatusEvent() (Applies status changes to DB)
├── lib/outreach/
│   ├── governor.ts     <-- Rate limiter (30-50 invites/day, max 180/week, jitter)
│   └── driver.unipile.ts <-- Unipile messaging & invite driver
└── client/src/routes/
    ├── api.unipile.webhook.$token.tsx <-- Handles BOTH notify_url & account_status webhooks
    ├── api.unipile.connect.tsx        <-- Authenticated API endpoint to mint auth link
    ├── recruiter.settings.tsx         <-- Recruiter Integrations UI Component
    └── owner.settings.tsx             <-- Sundar's Owner Seat & Subscription Management Panel
  
```

6. Database Schema Mapping (Project_Beacon_DB_schema_V2.prisma)

6.1 Already Existing in Schema

- User model (with OWNER, RECRUITER, CONTRACTOR roles).
- SystemConfig model (key, value, notes — stores unipile.max_slots).
- InteractionChannel.LINKEDIN_DM enum value.
- ConversationChannel enum (LINKEDIN, INSTAGRAM, WHATSAPP, SMS).
- InteractionEvent.deliveryStatus field.
- RecruiterScoreSnapshot.excludedMetrics (excluding degraded account windows from recruiter SLAs).

6.2 Models & Enums to Add to Prisma Schema

Add the following block to server/Project_Beacon_DB_schema_V2.prisma :

```
// =====
// UNIPILE INTEGRATION MODELS & ENUMS
// =====

enum AccountProvider {
    LINKEDIN
    GOOGLE
    OUTLOOK
    MAIL
    WHATSAPP
    INSTAGRAM
    TELEGRAM
    MESSENGER
    TWITTER
}

enum AccountStatus {
    CONNECTING          // CREATION_SUCCESS, CONNECTING
    OK                  // OK, SYNC_SUCCESS, RECONNECTED
    RECONNECTION_NEEDED // CREDENTIALS
    PERMISSION_REVOKED  // PERMISSIONS
    CONNECT_FAILED      // CREATION_FAIL
    SYNC_STOPPED        // ERROR, STOPPED, unmapped strings
    DISCONNECTED        // DELETED
}

model ConnectedAccount {
    id                String          @id @default(uuid())
    userId            String          @map("user_id")
    user              User            @relation(fields: [userId], references: [id], onDelete: Cascade)
    unipileAccountId String          @unique @map("unipile_account_id")
    provider          AccountProvider
    accountEmail      String?         @map("account_email")
    accountName       String?         @map("account_name")
    status            AccountStatus   @default(CONNECTING)
    rawUnipileStatus  String?         @map("raw_unipile_status")
    statusChangedAt   DateTime?       @map("status_changed_at")
    providerUserId    String?         @map("provider_user_id")
    publicIdentifier  String?         @map("public_identifier")
    isPremium         Boolean         @default(false) @map("is_premium")
    lastSyncedAt      DateTime?       @map("last_synced_at")
    lastEventAt       DateTime?       @map("last_event_at")

    // Rate governance counters
    invitesSentToday   Int             @default(0) @map("invites_sent_today")
    invitesSentThisWeek Int          @default(0) @map("invites_sent_this_week")
    countersResetAt   DateTime?       @map("counters_reset_at")
    weekStartedAt     DateTime?       @map("week_started_at")
    warmupStartedAt   DateTime?       @map("warmup_started_at")
    pausedUntil       DateTime?       @map("paused_until")

    createdAt         DateTime        @default(now()) @map("created_at")
    updatedAt         DateTime        @default(now()) @map("updated_at")

    degradations      AccountDegradation[]

    @@unique([userId, provider])
    @@index([status])
    @@index([provider, status])
    @@map("connected_accounts")
}

model UnipileAuthAttempt {
    id                String          @id @default(uuid())
    nonce            String          @unique
    userId           String          @map("user_id")
    provider         AccountProvider
}
```

```

kind          String          // "create" | "reconnect"
reconnectFor  String?         @map("reconnect_for")
expiresAt     DateTime         @map("expires_at")
consumedAt    DateTime?       @map("consumed_at")
createdAt     DateTime         @default(now()) @map("created_at")

@@index([userId])
@@index([expiresAt])
@@map("unipile_auth_attempts")
}

model UnipileWebhookEvent {
  id          String          @id @default(uuid())
  dedupeKey    String          @unique @map("dedupe_key")
  source       String          // account_status | messaging | users
  unipileAccountId String?     @map("unipile_account_id")
  payload      Json
  receivedAt   DateTime        @default(now()) @map("received_at")
  processedAt  DateTime?       @map("processed_at")
  processError String?         @map("process_error")

  @@index([source, receivedAt])
  @@index([processedAt])
  @@map("unipile_webhook_events")
}

model AccountDegradation {
  id          String          @id @default(uuid())
  connectedAccountId String    @map("connected_account_id")
  connectedAccount ConnectedAccount @relation(fields: [connectedAccountId], references: [id], onDelete: Cascade)
  status      AccountStatus
  startedAt   DateTime         @map("started_at")
  endedAt     DateTime?        @map("ended_at")

  @@index([connectedAccountId, startedAt])
  @@map("account_degradations")
}

```

7. Account Safety & Rate Governance

All outbound operations must pass through `server/lib/outreach/governor.ts` to ensure LinkedIn accounts remain safe:

Action	Vendor Limit	App Governor Safe Cap	Pacing Strategy
Invitations	80–100 / day (~200/week)	30–50 / day (180 / week max)	4–22 min randomized send delay
Profile Fetches	~100 / day (Standard)	80 / day cap	Cache profiles; refresh prior to send
Search Queries	1,000 / query cap	1,000 profiles / query	Facet search by location & industry
Circuit Breaker	On 429 / 422 HTTP response	Pause account 24h	Raise Escalation alert for Owner

8. Implementation Phasing

- **Phase 0 (Setup):** Provision Staging & Prod Unipile DSNs, verify Worker `?port=` workaround, execute GDPR DPA.
- **Phase 1 (Auth Pipeline):** Run database migration, deploy Hosted Auth endpoints, build dual-shape webhook handler, deploy Recruiter & Owner Settings UI.
- **Phase 2 (Inbound Messages):** Deploy messaging webhook handler, handle outbound echo filtering (`account_info.user_id === sender.attendee_provider_id`).
- **Phase 3 (Outbound Engine):** Deploy `governor.ts` , send invitations with notes for instant acceptance tracking, activate circuit breaker.